Data Mining Assignment II

Group 2

Shyaka Ivan

August 2026

Question 4 (Chapter 4: The Curse of Dimensionality)

Investigating the performance deterioration of local approaches like KNN when the number of features p becomes large.

(a) Single Feature Scenario ($p = 1$)

When X is uniformly distributed on the interval $[0, 1]$, any evaluation window of length 0.10 covers exactly 10% of the absolute range. Therefore, on average, the fraction of available observations used to make the prediction is:

$$\text{Fraction}_{p=1} = 0.10 \text{ (or } 10\%)$$

(b) Two Feature Scenario ($p = 2$)

Assuming independence between the two uniformly distributed features X_1 and X_2, the probability of an observation falling into a regional sub-space is the product of its individual dimensional probabilities. To be within 10% of the range for both features simultaneously, the fraction evaluates to:

$$\text{Fraction}_{p=2} = 0.10 \times 0.10 = 0.10^2 = 0.01 \text{ (or } 1\%)$$

(c) High-Dimensional Scenario ($p = 100$)

Extending the independent fractional product logic across a hypercube spanning 100 unique feature dimensions yields:

$$\text{Fraction}_{p=100} = 0.10^{100} = 1 \times 10^{-100} \approx 0$$

On average, the fraction of available data points sitting within this neighbor radius is practically zero.

(d) Drawback of KNN in High Dimensions

The mathematical outcomes of parts (a) through (c) demonstrate that as the feature space expansion coefficient p scales upwards, the data volume sitting within a fixed localized radius shrinks exponentially toward zero. Consequently, a core drawback of KNN in high-dimensional spaces is that **there are virtually no training observations "near" any given test target**. To construct a prediction, the algorithm is forced to expand its search radius and rely on highly distant data points, causing the "local" assumption of KNN to break down entirely.

(e) Hypercube Side Length Analysis

To construct a p-dimensional hypercube that captures a constant target volume fraction of 10% (0.10), the length of each side s is governed by the equation $s^p = 0.10$, which implies $s = \sqrt[p]{0.10}$.

- **For $p = 1$:** $s = 0.10^1 = 0.1000$ (The side covers 10% of the feature range)
- **For $p = 2$:** $s = 0.10^{1/2} \approx 0.3162$ (The sides must expand to cover 31.62% of each range)
- **For $p = 100$:** $s = 0.10^{1/100} \approx 0.9772$ (The sides must expand to cover 97.72% of each range)

Commentary: This reveals the core mechanics of the curse of dimensionality. In a 100-dimensional space, to capture just a tiny 10% fraction of your dataset, the tracking hypercube can no longer be considered "local." Its sides must stretch across nearly the entire available spectrum (97.72%) of every single feature dimension, defeating the purpose of a neighborhood-based approach.

Question 9 (Chapter 4: Odds and Probabilities)

Translating credit card default risk between the mathematical frameworks of probability and odds.

(a) Converting Odds to Fraction of Defaults

The mathematical relationship used to calculate a probability (p) from a given value of odds is defined as:

$$p = \frac{\text{Odds}}{1 + \text{Odds}}$$

Given that the baseline odds of defaulting on a credit card payment are equal to 0.37, substituting this parameter into our translation equation yields:

$$p = \frac{0.37}{1 + 0.37} = \frac{0.37}{1.37} \approx 0.2701$$

Conclusion: On average, a fraction of **0.2701** (or 27.01%) of the individuals within this group will in fact default on their credit card payments.

(b) Converting Default Probability to Odds

The mathematical relationship used to calculate odds from a known probability parameter (p) is defined as the ratio of success to failure:

$$\text{Odds} = \frac{p}{1 - p}$$

Given that an individual carries a 16% chance of defaulting ($p = 0.16$), the dry or non-default probability equates to $1 - 0.16 = 0.84$. Substituting these inputs into the ratio expression yields:

$$\text{Odds} = \frac{0.16}{1 - 0.16} = \frac{0.16}{0.84} = \frac{4}{21} \approx 0.1905$$

Conclusion: The absolute odds that this individual will default on her credit card payment evaluate to **0.1905** (or a ratio of $4 : 21$).

Question 14 (Chapter 4: Classification Tournament)

Developing and evaluating multi-class classification models (LDA, QDA, Logistic Regression, Naive Bayes, and KNN) to classify whether a vehicle achieves high or low fuel economy based on structural attributes.

(a) Target Feature Engineering (`mpg01`)

The binary response indicator vector `mpg01` was engineered by comparing individual vehicle fuel efficiency profiles against the sample median value (22.75). Observations matching values strictly above the baseline threshold are flagged as 1 (High Mileage), while metrics tracking below or equal to the median marker are designated as 0 (Low Mileage).

(b) Exploratory Data Insights

Bivariate boxplots were constructed inside the notebook file to evaluate cross-variable behaviors against `mpg01`. Group-stratified analysis confirms strong visual segregation patterns:

- Vehicles classified under high fuel efficiency (`mpg01` $= 1$) compress tightly within low weight regions, small-displacement profiles, and low cylinder configurations.
- Low mileage vehicles (`mpg01` $= 0$) exhibit heavy structural weights and high horsepower requirements.

Based on these distinct categorical divisions, four attributes are selected as the optimal modeling inputs: `cylinders`, `displacement`, `horsepower`, and `weight`.

(c) Data Validation Splitting

The filtered tracking dataset was partitioned into an 80/20% configuration using a fixed random seed. The larger slice forms the training matrix used to initialize algorithm boundaries, while the remaining 20% acts as the isolated test validation block.

(d), (e), (f), (g) & (h) Classification Model Performance Evaluation

Each algorithm was trained on the training matrices and checked blindly against the test observations. The compiled test error rates are summarized in the comparative layout below:

KNN Tuning Summary: Iterating across multiple neighborhood horizons shows that small values of K ($K = 1, 3, 5$) provide the best local fit on this data geometry. As the boundary scale parameter K scales past 10, neighbor

Classification Algorithm	**Test Error Rate**	**Decision Boundary Evaluation**
Linear Discriminant Analysis (LDA)	$\approx 11.39\%$	Robust execution using a linear hyperplane
Quadratic Discriminant Analysis (QDA)	$\approx 10.13\%$	Optimized fit utilizing curved boundaries
Logistic Regression	$\approx 11.39\%$	Identical performance to the LDA linear ap
Naive Bayes (Gaussian)	$\approx 10.13\%$	Strong result matching QDA error baselines
K-Nearest Neighbors (KNN)	$\approx 11.39\%$	Peak accuracy localized around tuning para

Table 1: Classification Tournament Evaluation Performance Matrix

boundaries over-smooth, leading to increased classification mistakes on unseen test points.

Question 15 (Chapter 4: Functional Programming and Visualization)

This problem involves constructing programmatic functions to evaluate exponential expressions and display their geometric behaviors using modular visualization scripts.

(a), (b), (c) & (d) Summary of Custom Functions

The custom functions `Power()`, `Power2()`, and `Power3()` were successfully implemented within the accompanying Jupyter Notebook file.

- `Power()` explicitly computes and prints the evaluation of $2^3 = 8$.
- `Power2(x, a)` generalizes this process to dynamically print any base x raised to power a. Executing this module yields:
  - $10^3 = 1,000$
  - $8^{17} = 2,251,799,813,685,248$
  - $13^{13} = 302,875,106,592,253$
- `Power3(x, a)` updates this operational flow by natively returning the value as a reusable Python object rather than merely printing it to the console buffer, allowing the results to be saved into arrays for downstream visualization steps.

(e) Geometric Scaling and Logarithmic Transformations

Using the objects returned from `Power3()`, the exponential function $f(x) = x^2$ was mapped across a sequence domain of integers from 1 to 10.

Analysis of Coordinate Scale Alterations: When plotted on a standard linear Cartesian plane, the function $f(x) = x^2$ displays an upward-sweeping parabolic curve reflecting classical geometric growth. However, when both axes are configured to a logarithmic scale using the `.set_xscale('log')` and `.set_yscale('log')` method transformations, the trajectory flattens into a perfectly straight linear vector.

This behavior occurs because applying standard logarithms to power-law models transforms exponential multiplication components into basic linear configurations:

$$\log(y) = \log(x^2) \implies \log(y) = 2 \cdot \log(x)$$

On a log-log coordinate grid, this system behaves exactly like a standard line ($Y = mX + C$), where the resulting slope coefficient (m) corresponds perfectly to the static mathematical exponent (2).

(f) Implementation of the Generalized Plotting Tool

The generalized wrapper function `PlotPower()` was compiled inside the code notebook. When executed with the argument criteria `PlotPower(np.arange(1, 11), 3)`, the function systematically constructs an array sequence spanning from 1 to 10, calculates their corresponding cubed values $(1^3, 2^3, \ldots, 10^3)$, and automatically outputs a labeled bivariate trend plot mapping the relationship.